Linux

BASIC COMMANDS

==========

#ls

--> will display contents of the present working directory

#ls -l (l for long list)

--> will display properties/attributes of all contents

#ls -l install.log --> to see a single file properties

#ls -ld Desktop --> to see a directory properties

#ls -lt

--> will all contents according to time stamp (date and time)

#ls -ltr

--> reverse time stamp

#ls -iltr

#ls -R /root --> list the subdirectories recursively

Visual Classification of Files With Special Characters Using ls -F

Instead of doing the ‘ls -l’ and then the checking for the first character to determine the type of file. You can use -F which classifies the file with different special character for different kind of files.

$ ls -F

Desktop/ Documents/ Ubuntu-App@ firstfile Music/ Public/ Templates/

Thus in the above output,

/ – directory.

nothing – normal file.

@ – link file.

* – Executable file

Visual Classification of Files With Colors Using ls --color=auto

---

Recognizing the file type by the color in which it gets displayed is an another kind in classification of file. In the above output directories get displayed in blue, soft links get displayed in green, and ordinary files gets displayed in default color.

$ ls --color=auto

Desktop Documents Examples firstfile Music Pictures Public Templates Videos

#pwd

or

#echo $PWD

#cd

#cd ..

#cd -

#date

#date -s "25 Nov 2012 10:12:16"

#date +%Y%m%d%T -s " 2012-12-23 12:16:23"

#cal --> displays present month calender

#cal -j --> Display Julian dates (days one-based, numbered from January 1)

#cal -y --> Display a calendar for the current year.

#cal -m --> Display Monday as the first day of the week.

#cal -s --> Display Sunday as the first day of the week.

#cal -3 --> Display prev/current/next month output.

#cal 9 2040 --> displays 9th month of 2040 year.

TOUCH COMMAND:

==========

We use touch command for

1. To create empty(zero size) files

2. To create multiple zero size files in the single command

3. To change the "time stamp" of a file/dir

NOTE: normal users also can change the time stamp of a files.

CAT COMMAND:

==========

we use cat command for

1. To create files

2. To read already created files

3. to append some text to a already created file

4. To copy multiple files contents into a single file

#cat > <filename>

--> enter and ctrl+d to save

#cat <filename> --> to read a file

#

#cat > /etc/laxma

#touch /tmp/file{1..6}

#cat >> <filename>

--> to append into already created file.

#cat file1 file2 > file3 (to copy file1, file2 contents into file3)

=> if file3 is already existing it will overwrite , otherwise it will create a file with the name file3 and copy the contents of file1 file3 into file3

#rm <filename> --> to remove a file interactively

#rm -f <filename> --> to remove a file forcefully

#mkdir <dirname> --> to create a directory

#rmdir <dirname> --> to delete a empty direcotry.

#cd /redhat

#touch file{1..6}

#rm -r <diranme> --> to remove a directory contents in a interactive mode.

#rm -rf <dirname> --> to remove a directory forcefully which is having contents

rm -r file1 dir1 Delete file1 and dir1 and its contents.

rm -rf file1 dir1 Same as above, except that if either file1 or dir1 does not exist, rm will continue silently.

* All files

g* Any file begining with g

b*.txt Any file begining with b followd by any characters and ending with .txt

data??? Any file begining with data followd by exactly three characters

[abc]* Any file begining with either a, b, or c

backup.[0-9][0-9][0-9] Any file begining with backup followed by exactly three numerals

[[:upper:]]* Any file begining with an uppercase letter

[![:digit:]]* Any file not begining with a numeral

rm *.html

COLLABORATIVE DIRECTORIES

==========

#mkdir dir1 dir2 dir3

#mkdir /nfs/root/directory

Note: if /nfs/root is already existing then above command gets execute otherwise it throws "mkdir: cannot create directory `/nfs/root/directory': No such file or directory" error.

#mkdir -p /nfs/root/directory

#mkdir -p COSS/{RHCE/{rhcsa,rhce},RHCVA{111,222},RHCSS/{rh423,rhs429,rhs333}}

NOTE: -p is the option to create the collaboratvie directories

#yum install tree* -y

#tree COSS

OR

#ls -R COSS

#cp file1 file2 --> copy file to file2 , if file2 exists, it is overwritten with the contents of file1. if file2 done not exist, it is created.

#cp -i file1 file2 --> Same a above, except that if file2 exists, the user is prompted before it is overwritten

#cp file1 file2 dir1 --> Copy file1 and file2 into directory dir1. dir1 must already exist.

#cp fruits.sort backup

#cp -p fruits.sort backup.1 --> it will copy along with the time stamp

#cp -rfvp <directory> <backupdirname>

NOTE: r --> recursively ,

f --> forcefully ,

v --> verbose,

p --> preserve

MV COMMAND:

==========

#mv file1 file2 --> move file1 to file2 . If file2 exists, it is overwritten with the contents of file1 . If file2 doesn't exist. it is created .

#mv file1 file2 dir1 --> Move file1 and file2 into directory dir1. dir1 must already exist.

#mv dir1 dir2 -->Move directory dir1 (and its contents) into directory dir2. If directory dir2 does not exist, create directory dir2, move the contents of directory dir1 into dir2, and

delete directory dir1.

#mv fruits fruits.main

#mv fruits.main marchsalesreport (it will overwrite)

or

#rename <oldname> newname <oldname>

#nl install.log --> to open a file with line numbers

#more install.log --> if any file is size of more than one page , then if you want to open that file page wise

#nl install.log | more

#nl install.log | less (provides the facility to come back to the previous pages which is not possible in more command)

#pr install.log --> shows the number of pages are there in the install.log file.

#

tail /etc/passwd --> prints/shows last ten lines

head /etc/passwd --> first

#tail -3 /etc/passwd --> prints last 3 lines

#head -3 /etc/passwd --> prints first 3 lines

#

#sort filename

#sort filename > filename.sort

#

#touch laxma ; mv laxma laxma.renamed

--> the above command creates laxma file and renames it as laxma.renamed, onething we have to notice here that is we are running multiple command is single command.

#shutdown -h 30 --> shutdowns after 30 mins

to cancel the shutdown process

#ctrl+c

#ps -ef | grep shutdown

#kill <pid of shutdown>

#shutdown -s -t 3000 --> shutdowns windows 3000 secons

#tput setf 2

#tput setf 3

#tput setf 4

#tput reset

how to find when a linux system is last rebooted? who rebooted?

#last reboot

or

#who -b

#last --> to see the user name who rebooted lastly.

#logname --> the logname(login name) command show the name of the real user who logs in initially. If that user uses the su command to switch identity, the logname command, unlike the whoami command, whill still show the real username.

#uname --> the uname command produces basic information about the system. without any options , this command displays the operating system name only.

#cat /etc/redhat-release

#clear

--> the clear command clears the terminal screen and places the cursor at the begining of the screen.

LINUX FILE SYSTEM HIERARCHY

==========

Note: # man hier --> to see the manual page for file system hierarchy in linux

/root:

======

- This is the default home directory of the root user.

/home:

======

- This is the home directory for all Normal/Standard users

- When ever any normal user logs in he gets access to his home directory (/home/<username>)

#cd /home/

#ls

/boot:

======

- It contains all the necessary files related for booting the OS such as bootloader.

- It also contains the kernel which core of the OS.

/sbin ,/usr/sbin:

==========

- stands for system binary

- /sbin & /usr/sbin will contains all the essential commands which can only be executed by the root user

example: fdisk , useradd, service, ifup, lvremove, lvcreate etc.

#which useradd

#which service

#which ifup

#which fdisk

/bin ,/usr/bin:

==========

- bin stands for binary.

- /bin and /usr/bin directories will contains binaries/commands which be executed by all users.

example: ping, cat, vi, ls, pwd, hostname, passwd etc

#which cat

#which pwd

#which hostname

#which ping

/media:

=======

-This directory contains mount points for removable media such as CD and DVD disks or

USB sticks

/mnt:

=====

- This directory is a mount point for a temporarily mounted file systems.

/etc:

=====

- stands for et cetra

- It contains all the system configuration files of local system

exampls:

=======

vsftp --> /etc/vsftpd/vsftpd.conf

samba --> /etc/samba/smb.conf

nfs --> /etc/sysconig/nfs

/etc/exports

/var:

====

- var stands for variable

- This directory contains files which may change in size, such as spool and log files and websites and ftp sites.

example: # cd /var/log --> main logs directory

cat /var/spool/mail/test --> mail db file for test user

cd /var/ftp/pub --> document root for ftp server

cd /var/www/html --> document root for apache application.

/tmp:

=====

- stands for temporary

- This directory contains temporary files which may be deleted with no notice, such as by a regular job or at system boot up

- default duration will be 10 days.

- /var/tmp those files and directories will be deleted after 30 days.

#vim /etc/cron.daily/tmpwatch --> to see script which will delete automactically.

/net:

=====

- /net is a virtual directory which will be created by the autofs deamon if it is running.

- /net/<nfs serverip> directoy give direct access to the NFS serve shares without mounting the NETWORK shares manually .

#cd /net/192.168.0.254

/dev:

=====

- dev stands for devices

- It contains information about all Hardware Devices

examples: /dev/sda (/dev/sda1, /dev/sda2 etc)

#tty

/opt:

=====

- opt stands for optional
- This directory should contain add-on packages that contain static files.

- It generally contains thrid party softwares like openoffice, kaspersky Antivirus.

/usr:

=====

- stands for Unix System Resources
- It contains the programs & applications which are available for users
- similar to programs files in Windows.

#yum install vsftp* -y

firefox /usr/share/doc/vsftpd-2.2.2/ &

#yum install samba* -y

#firefox /usr/share/doc/samba-common-3.5.10/ &

The /proc File System

The /proc file system is a special file system that is generated in system memory. It does

not exist on any disk. /proc contains files that provide important information about the state of your system. For example, /proc/cpuinfo holds information about your computer's CPU processor. /proc/devices lists those devices currently configured to run with your kernel. /proc/filesystems lists the file systems. /proc files are really interfaces to the kernel,obtaining information from the kernel about your system. Table 30-5 lists the /proc subdirectories and files. Like any file system, /proc has to be mounted. The /etc/fstab file will have a specialentry for /proc with a file system type of proc and no device specified.none /proc proc defaults 0 0

#cat /proc/num :There is a directory for each process labeled by its number.

example: #cd /proc/1 --> is the directory for process 1.

#cat /proc/cpuinfo -->Contains information about the CPU, such as its type, make, model, and performance.

#cat /proc/devices --> Lists the device drivers configured for the currently running kernel.

#cat /proc/loadavg -->Lists the system load average.

#cat /proc/meminfo --> Displays memory usage.

#cat /proc/modules -->Lists the kernel modules currently loaded.

#cat /proc/uptime --> Displays the time the system has been up.

#cat /proc/version --> Displays the kernel version

/proc/partitions

//putta.ramireddy@gmail.com

#cat > /tmp/file.txt

#mkdir /tmp/test/

#cat > /tmp/test/sample.txt

#rm -f /tmp/test/sample.txt

#rm -rf /tmp/test

Absolute Paths And Relative paths

==========

Absolute Path Names	Relative path Names
1. Path name begins with a slash (/)	1. Doesn't begin with a slash
2. Uses slashes to separate directories in the path names.	2 --do--

3. Does not change unless the file is moved. 3. changes according to the present working

directory.

example: For Absolute path

========

#vim /usr/share/doc/samba-common-3.5.4/README

exaples: For Relative paths

#vim share/doc/samba-common-3.5.4/README

#vim doc/samba-common-3.5.4/README

Secure Linux File Access

==========

r - Read

w - Write

x - execute

OBJECTIVES:

==========

1. User,Group,Other (UGO) Concepts

2. Manage permissions from the command line

=> In this unit you will learn how to manage the ownership and permissions of files and directories from command line.

Why is file security important on multiuser system?

1. Prevent unauthorized access and modification by unauthorized users.

2. Permit collaboration among users working on a common task.

#key points

there are also three basic permissions for files and directories; read , write, execute.

absolute mode/numeric mode symbolic mode

UGO permissions

==========

FIle Permissions Directory Permissions

---------- ----------

r,4 = read, view r = list contents

w,2 = write, update w = create/delete contents

x,1 = execute. x = access(enter into the directory)

- = a permission isn't set - = a permission isn't set

ls -l <file name> --> to see any file attributes/properties

#ls -ld <dir name> --> to see any directory attributes/properties

#ls -l install.log

-rw-r--r--. 1 root root 42048 Oct 26 10:36 install.log

- --> indicates type of file

there are total 7 types of files

==========

1. - --> indicates Regular file

2. d --> indicates directory

3. c --> indicates character file

4. b --> indicates block speical file

5. l --> indicates link file

6. s --> indicates socket file }

7. p --> indicates named pipe } IPC (inter process communication)

Link files:

==========

link file are like short cuts in windows, to access the same file in multiple locations we use link files. It is also called as "soft link" or "symbolic link"

#ln -s laxma.txt /opt

#ls -l /opt/laxma.txt   (to see type of file)

IN GUI

======

#ctrl+shift+drag&drop

OR

right click and makelink

BLOCK FILE: Device files can be classified as "character special" or "block special files"

Character special:

==========

character special files represent devices that interact with linux character-by-character

ex: terminal, printers

#tty

#ls -l /dev/pts/1

Block special files:

==========

Block speical files represent devices such as hard disk or pendrives which interact with linux using bloks of data.

#ls -l /dev/sda

#ls -l /dev/sdb1

NAMED PIPE FILE:

==========

A named pipe allows two unrelated processes running on the same system or on two different systems to communicate with each other and exchange data. Named pipes are uni-directional.

They are also referred to as FIFO because they use First in First Out mechanism. Named pipes make inter process communication(IPC).

#find / -type p --> to find the named pipe files

#

#mknod -m -t p p --> to create a named pipe

#mv p laxman

OR

#mkfifo laxma

#ls -l laxma

Socket Files:

==========

A socket is a named pipe that works in both directions. IN Other words, a socket is a two-way named pipe. It is also a type of IPC. Sockets are used by client/server programs.

rw-r--r--

=========

rw- --> permissions belongs "owner" of the file

r-- --> permissions belongs to "group" owner of the file

r-- --> permissions belongs to "others"

. --> means no acl

+ --> means acl is applied

#ls -ld <dirname> --> to see attributes of a directory.

Permission can be edited in two ways

1. absolute mode

2. symbolic mode

rwxr-xr-x = 755

rw-r--r-- = 644

==>chmod is the command line tool available to change the file/directory permissions

chown --> to change the ownership of a file/directory

chgrp --> to change the group ownership of a file

chmod syntax:

==========

#chmod whowhatwhich file|directory

who =u,g,o,a

what =+,-,=

which =r,w,x

symbolic mode

NOTE: u --> owner, g--> group, o --> others (student,visitor etc)

#chmod u-w,g+w,o+w install.log

#chmod u=rw,g=r,o=r install.log

#chmod a=w install.log

--w--w--w- 1 root root 13 Feb 19 11:23 install.log

#chmod a+rwx install.log

-rwxrwxrwx 1 root root 13 Feb 19 11:23 install.log

---

absolute mode/Numeric mode

#chmod 466 install.log

#chmod 666 install.log

Note: + --> add

- --> remove

= --> override

#chown --> to change the ownership of a file/directory

user: rps ---> install.log rwxr--rwx

user : student

#chown student install.log

#cat > /test.txt

#chown student /test.txt --> ownership of file to student

#chgrp student /redhat

#chown student:student /test.txt --> owner&group ownership to student

##

#chgrp -R student /redhat

#chown student:student /redhat/

#chown -R student:student /redhat --> recursively changes owner and group owner of the directory.

NOTE : IF ROOT CREATES A FILE & DIRECTORY

644 755

IF NORMAL USER CREATES A FILE & DIRECTORY DEFAULT PEMISSIONS WILL BE

664 775

REASON: deafault permission = full permissions - umask

to change umask temporarly #umask 0002

vim /etc/bashrc (to change permanently)

#vim /etc/profile

PRACTICE QUIZ

==========

Practice Quiz

Linux User, Group, Other Concepts

Answer the True/False Questions based on the following user and file congfigurations.

---

users and their groups

lucy lucy,ricardo

ricky ricky,ricardo

ethel ethel,mertz

fred fred,mertz

File attributes (permissions, user & group ownership, name):

drwxrwxr-x ricky ricardo dir (which contains the following files)

-rw-rw-r-- lucy lucy lfile1

-rw-r--rw- lucy ricardo lfile2

-rw-rw-r-- ricky ricardo rfile1

-rw-r----- ricky ricardo rfile2

Questions regarding the lfile1 file.

1. lucy can change the contents of lfile1

   (select one of the following)

   a. True

   b. False

2. fred can change the contents of lfile1

   (select one of the following)

   a. True

   b. False

3. fred can delete lfile1

   (select one of the following)

   a. True

   b. False

4. ricky can change the contents of lfile1

   (select one of the following)

   a.True

   b.False

5. ricky can delete lfile1

   (select one of the follwing)

   a.True

   b.False

Users and their groups:

lucy lucy,ricardo

ricky ricky,ricardo

ethel ethel,mertz

fred fred,mertz

File attributes(permissions, user & group ownership, name):

drwxrwxr-x ricky ricardo dir(which contains the following files)

-rw-rw-r-- lucy lucy lfile1

-rw-r--rw- lucy ricardo lfile2

-rw-rw-r-- ricky ricardo rfile1

-rw-r----- ricky ricardo rfile2

Questions regarding the lfile2 file:

1. ricky can view the contents of lfile2

(select one of the following)

a. True

b. False

2. ricky can change the contents of lfile2

(select one of the following)

a.True

b.False

3. ricky can delete lfile2

(select one of the following)

a.True

b.False

4. ethel can view the contents of lfile2

(select one of the following)

a. True

b.False

5. ethel can chage the contents of lfile2

(select one of the following...)

a. True

b. False

Users and their groups:

lucy lucy,ricardo

ricky ricky,ricardo

ethel ethel,mertz

fred fred,mertz

File attributes (permissions, user & group ownership, name):

drwxrwxr-x rikcy ricardo dir(which contains the following files)

-rw-rw-r-- lucy lucy lfile1

-rw-r--rw- lucy ricardo lfile2

-rw-rw-r-- rikcy ricardo rfile1

-rw-r----- ricky ricardo rfile2

Questions regarding the rfile1 file:

1. Lucy can view the contents of rfile1

   (select one of the following...)

   a.True

   b.False

2. lucy can change the contents of rfile1

   (select one of the following ...)

   a.True

   b.False

3. fred can view the contents of rfile1

   (select one of the following...)

   a.True

   b.False

4. fred can change the contents of rfile1

   (select one of the following...)

   a.True

   b.False

VIM EDITOR

==========

VIM(vi improved) is a powerful text editor , The vim editor supports sophisticated text manipulations which are very useful for system administration. If you are familiar with the vi utility, you will find that vim includes vi features, plus many other features.

VIM editor has three modes of operations.

1.Command Mode

2.Insert Mode

3.Ex Mode(Extended Command Mode)

COMMAND MODE:

==========

After opening the vim editor, you begin in command mode. The basic operations we can perform in command mode are like , copy, paste, delete, undo ,redo,search, replace , to go to 1st line and end line of the file etc.

dd --> Deletes a line

ndd --> Deletes 'n' lines

yy --> copies a line

nyy --> Copies 'n' lines

p --> pastes the copied or deleted text

u --> Undo (you can undo 1000 times)

ctrl+r --> Redo

G --> Moves the cursor to the last line of the file

gg --> Moves the cursor to the begining line of the file

shift+zz --> to save and quit the file (this will not work in case of forcefull close)

/<word> --> to search any word

(press n for next match and N for previous match)

#vim +/http /etc/services --> to search for http word directly from bash prompt.

5yw --> to copy 5 words

5cw --> to cut 5 words

5dw --> to delete 5 words

crtl+v --> visual mode

k

h l

j

h --> to move left side one character

l --> to move right side one character

k --> to move cursor above the character

j --> to move the cursor below the character --> in AIX arrows will not suppoted

INSERT MODE:

==========

In insert mode you can begin entering text by pressing (i,I,a,A,o,O) any one. You can use arrow keys to move the cursor in vim while remaining in insert mode. Press ESC key to leave insert and return to command mode.

i --> Inserts the text before the current cursor position

I --> Inserts the text in begining of a line

a --> Adds the text after the current cursor position

A --> Adds the text at the end of a line

o --> Inserts the text one line below current cursor position

O --> Inserts the text one line above current cursor position.

Ex Mode:

========

From command mode , press a : character to go into ex mode and the cursor moves to the bottom of the screen.

:q --> Quit without saving

:q! --> Quit forcefully without saving

:w --> write (save)

:wq --> save and quit

:wq! --> save and quit forcefully

:se nu (or) set nu --> sets line numbers

:se nu (or) set nonu --> to remove the line number from vim editor

:84 --> the cursor goes to line 84

:set key=<passwd> ---> to encrypt the file

:set key= enter ---> to decrypt the file

:%s/bin/bash/ ---> We want to search for all bin words in the entire file and replace them with bash

:%s/bin/bash/gi here g means --> globally, I for --> case-insensitive.

:45s/bin/bash ---> "bin" will be replaced with "bash" in 45th line

:45,48s/bin/bash ---> in 45th to 48th lines "bin" will be replaced with "bash"

:w filename --> original file will be saved as the given filename , like save as option.

:2,15w test.txt --> creates a new file with the name "test.txt" with the text of 2-15 lines

:x --> to save and quit the file in EX-mode

:X --> to encrypt the file in EX-mode

:11,15d --> to delete 11th line to 15th lines

:.,13 --> deteles from current/present cursor line to 13th lines.

#vim -o /etc/fstab /etc/group (to open multiple files horizontally)

#vim -O /etc/fstab /etc/group (to open multiple files virtically)

vim /etc/passwd (ctrl+w+s to create one more copy of a file in horizontally)

(ctrl+w+v vertically)

NOTE: ctrl+shift+ww --> to switch in between the two files

Nano Editor Commands

nano <filename> to open a file for editing or writing

save --> ctrl + O

exit --> ctrl + x

cut ---> ctrl + K

paste --> ctrl + U

search a text --> ctrl +w

goto line number --> ctrl + _ (number)

undo ---> Alt + U

redo --> Alt + E

Shell

A Command-Line Interpretor that connects a user to Operating System and allows to execute the commands or by creating text script.

Bash Shell Scripting

What is Shell Script ?

Normally shells are interactive. It means shell accept command from you (via

keyboard) and execute them. But if you use commands one by one (sequence of 'n' number of commands) ,then you can store this sequence of commands to text file and tell the shell to execute this text file instead of entering the commands.
This is know as shell script.

Why to Write Shell Script ?

==>Useful to create our own commands.
==>Save lots of time.
==>To automate some task of day-to-day life.
==>System Administration part can be also automated.

Other popular scripting language are perl, python, ruby along with bash.

Execute your script as
---
syntax:

#bash your-script-name
or
#sh your-script-name
or
#./your-script-name

BASH VARIABLES

==========

Generally you can find two types of variables

1. Environment variables
2. User defined variables.

=>simply we can say Environment variables are predefined system variables

=>when a user logins a bunch of environment variables are set for user. These variables control some basic functionality of your interaction with the shell.

#env --> env displays all environment variables for that user.

EXAMPELS:

=========

#echo $PWD

#echo $HOME

-->PWD,HOME both are predefined Environment Variable so when we use them with echo, we are saying go and get the value of HOME & PWD and print the output

on to the terminal, which are storing some value/text.

#cd /tmp

#echo $PWD --> here this command output will be /tmp"since we are using /tmp directory

#su - student --> this is to switch to "student" user form current user.

#echo $USER -->o/p will be student

to define our own ENV variables

==========

#vim /etc/profile

/export --> searching for "export"

LINUX=opensource --> here we are creating LINUX variable which is storing "opensource" value in it

export LINUX

#source /etc/profile --> to update the changes in /etc/profile file

#env | grep LINUX

#echo $LINUX

User defined variables:

==========

=> The below are used defined variables

#echo $sri --> o/p will be empty line bc "sri" is not a already defined variable

#sri=infotech

#echo $sri

#firstname=SRIinfotech

#secondname=offshore

#echo $firstname$secondname

SRIinfotech offshore

#read a --> Using "read" command we can take user inputs and assign that values to the variales to which we want.

#echo $a

to unset defined user variables

#unset a --> here a is a user defined variable name

to define variable persistently

#vim /etc/profile

---

a=10

Double quotes(") and single quotes(') behave differently

Double quotes(" "):

==========

=>Double quotes tells the bash shell go and get the value of the variables.

example:

root]#echo " The current user present working directory is $PWD "

o/p will be:

The current user present working directory is /root

root ~]#echo " Presently loggedin user is $USER "

o/p will be:

"Presently loggedin user is root"

Single quotes ('):

==========

=>single quotes tells the bash shell NOT to get the value of the variable.

=>single quotes are commonly used to allow a special characters to be part of a string.

so as a result it prints whatever is in between single quotes on to the terminal.

root#echo 'The current user present working directory is $PWD '

o/p will be:

The current user present working directory is $PWD

Don't get confuse with backtick(`) it is differen from single quotes(')

==========

=>backtick allows us to store any command output into a string , and then when you use it with $string, it prints the output of that command.

#space_used=`du -sh /home/student | awk '{ print $1}'`

#echo "student using $space_used of space "

---

student is using using 40K of space

---

wc command usage

---

wc displays number of lines, words and characters

[root@desktop57 ~]# wc /etc/fstab

19 90 913 /etc/fstab --> here 19 is the number of lines, 90 is the number of words, 913 is the number of characters of /etc/fstab file

[root@desktop57 ~]#

[root@desktop57 ~]#

[root@desktop57 ~]# wc -l /etc/fstab

19 /etc/fstab

[root@desktop57 ~]#

[root@desktop57 ~]#

[root@desktop57 ~]#

[root@desktop57 ~]# wc -w /etc/fstab

90 /etc/fstab

[root@desktop57 ~]#

[root@desktop57 ~]#

[root@desktop57 ~]# wc -c /etc/fstab

913 /etc/fstab

[root@desktop57 ~]#

[root@desktop57 ~]#

[root@desktop57 ~]#

[root@desktop57 ~]# wc -c /etc/fstab

913 /etc/fstab

COMMAND SUBSTITUTION

==========

We can user `` or $() to store a command output into a variable

[root@desktop57 ~]# linuecount=`wc  -l  /etc/fstab`

[root@desktop57 ~]#

[root@desktop57 ~]#

[root@desktop57 ~]# echo $linuecount

19 /etc/fstab

[root@desktop57 ~]#

or

#linecount=$(ls -l /root)

$echo "$linecount"

ALIASES

Using command aliases we can create costomized commands, aliases can be created for existing commands of non-existing commands.

#alias --> displays already existing alias commands

#sri --> you will get o/p as command not found.

#alias sri='ls -iltr'

#sri --> sri will display the output of "ls -iltr" , since sri is a alias command for ls -iltr

Note: above one is temporary, if you want to make it permanent to user specific , define it in ".bashrc" file

to remove it

#unalias sri

to make it permanent/persistent for all users define it in /etc/bashrc

---

alias sri='ls -iltr'

Understanding read command usage

==========

#read NAME --> here read command takes user input in interactive way.

laxmareddy

#echo $NAME

laxmareddy

#read -p "Enter the user name:" NAME

Enter the user name: laxmareddy "laxmareddy is user input"

#echo $NAME

laxmareddy

#vim addingnos.sh

#!/bin/bash

###A sample script which add two values and assigns it to another variable in non-interactivly.

```
a=10
echo "the value of a is $a"
b=20
echo "the value of b is $b"
c=$[ $a + $b ]

echo "the sum of a and b is $c"
```

OR

```
#!/bin/bash
## this script takes a and b variables values interactively and assigns it to another variable.
## -p with read means whatever  is given in b/w "" will be displayed while script execution, just for user unserstanding.
read -p "Enter the value of a :" a
read -p "Enter the value of b :" b

c=$[$a+$b]
echo "the sum of two nos is ${c}"
```

#chmod a+x addingos.sh (a for all users)

#./addingnos.sh (to execute)

DEBUGING THE BASH SHELL SCRIPT

==========

#bash -xv ./script-name --> debuging is very important , to findout in which line of the script error has occured , so that we can fix it quickly.

#echo $PATH

#addingnos.sh

=> if the above command "ays no such file or directory". if you want , the script should get executed even if we just give script name ,with out the absolute path.

for this move this "addingnos.sh " script to any one of the directories in $PATH, or create the addingno.sh script in a direcotry and add that directory to the

$PATH , then we can execute the script just by giving only script name.

#export PATH=$PATH:/root

#addingnos.sh --> now this script should get execute becuase /root has been included to $PATH

OR

#sh addingnos.sh

OR

#bash addingnos.sh

Conditional Execution

==========

|| = logical or

&& = logical and

EXAMPLES:

#id student && touch sample1

--> if first condition is true then it will execute 1st condition and it will go to 2nd command , executes it.

--> if 1st condition is false then it will not even check 2nd command.

#id student || touch sample2

#id laxma || touch laxma

--> if 1st condition is ture it will execute the 1st condition and it will skip 2nd

--> if 1nd condition is false it will omit the 1st condition and it will execute the 2nd condition.

FOR LOOP

=========

SYNTAX: for <variable> in list

do

command1

command2

....

done

Adding multiple user using FOR LOOP

==========

#vim addingusers

#!/bin/bash

#purpose to add multiple users into the system using for loop

for name in john max dax alice natasha

do

useradd $name

echo "redhat" | passwd --stdin $name

done

ADDING THE USERS WHOSE NAMES ARE IN A FILE

==========

#vim /opt/file

---

dax

max

jack

alice

natasha

#vim useraddscript3

```
for user in `cat /opt/file`
do
useradd $user
read -sp  "Enter the password for $name:" P
echo "$P" | passwd --stdin $user

echo "The user $user has been added successfuly"
done
```

#adding users with userinput :

==========

vim addinguser2

#adding the user and giving hiden passwd with read command.

```
read -p "Enter User's NAME: " u
useradd $u
read -sp " Enter user's passwd: " p
```

echo $p | passwd --stdin $u

echo "user $u added "

###IF ELSE CONDITION SYNTAX###

==========

###ifesle syntax

if <condition1>

then

command1

command2

.....

elif <condition2>

then

command1

command2

....

else

command1

....

fi

next_stat

TEST COMMAND []

==========

=> test command checks the truthfulness of a command.

Options:

-d - FILE exists and is a directory

-e - FILE exists

-f - FILE exists and is a Regular file

-s - FILE exists and has a size greater than zero

-h - FILE exists and is a symbolic link

-r - FILE exists and read permission is granted

-w - FILE exists and write permission is granted

-x - FILE exists and execute(or search) permission granted.

-a - and

-o - or

-b - FILE exists and is block special

-c - FILE exists and is character special

! - not

Numeric equality: STRING comparision

========== ==========

-eq - equls to ==

-ne - not equls to !=

-lt - less than <

-gt - greater than >

-le - less than or equals to <=

-ge - greater than or equls to >=

(EXPRESSION) --> expression is true

! EXPRESSION --> expression is false

EXPRESSION1 -a EXPRESSION2 --> Both EXPRESSION1 and EXPRESSION2 are true

EXPRESSION1 -o EXPRESSION2 --> either EXPRESSION1 or EXPRESSION2 is true

EXAMPLES OF TEST

==========

#test -d /tmp or [-d /tmp] --> it checks whether /tmp is a directory or not

#echo $? --> for checking exit status

NOTE: If exit status value is 0 , that means above command output is "successful/standard output" ,

if exit status is >0 , then the above command

output is "unsuccessful/error/"standard error""

#test -f laxma or [-f laxma]

#echo $?

#test -w laxma

#test -x laxma

==>write a shell script , which should prompt for user name ,

and take the user name interactively And script should check for wheather this user

alreay existing or not , if existing ,script should come out , if user is not existing

it should create the user with the password given by the user.

#vim laxmalinuxpro

```
read -p "Enter the  user name:" U
id $U
if test $? -gt 0
then
useradd $U
read -sp "Enter the user passwd:" P
echo "$P" | passwd --stdin $U
echo "the user $U has been added successfully"
else
echo "the user is already existing"
fi
```

UNDERSTANDING SEQ

==========

==> seq bydefault lists starting number to endnumber

example: # seq 1 10

---

o/p: 1 2 3 4 5 6 7 8 9 10

#seq 5 10

---

o/p: 5 6 7 8 9 10

#seq 10

---

1 2 3 4 5 6 7 8 9 10

#seq 1000 > junk1.data --> We can create a new file with some size

#ls -lh junk1.data

to show background running job example

#seq 10000000000 > junk1.data &

--> this command creates a file called "junk1.data" and keeps on increasing file size in the background. to view background running jobs

run below command

#jobs

#jobs -l

#watch -n 1 ls -lh junk1.data --> to view progress of the junk1.data file for each one second

#watch -n 1 df -TH --> refreshes df -TH output for every one second

The below script is useful to check how many systems are in network/pingable out of 1-15.

#vim networkcheck

#!/bin/bash

###this is the script to check the network

for ip in $(seq 1 15)

do

```
        ping  -c 1 192.168.0.$ip &> /dev/null
        if test $? -eq 0
        then
        echo "the ip 192.168.0.$ip is in network"
        else
        echo "the ip 192.168.0.$ip is not in network" >&2
        fi
 done

#/root/script7   >> cop     2>>  eop
#cat cop
#cat  eop
```

```
#jobs

#jobs -l

#ls  -lh  junk1.data

# watch  -ls -lh  junk1.data

 #df -TH
 # watch -n 1  df -TH                    ( show this with pendrive)

#vim testuser.sh
```

```
#!/bin/bash
# -o means or
# -eq means ==

if test "$USER" == "root" -o $(id -u) -eq 0
then
echo " the currently working user is root "
else
echo "the currently working user is  $USER "
fi
```

```
#sh testuser.sh
```

```
the user is root
```

#cp -p testuser.sh /   --> here we are copying testuser.sh script to / becuase on /root standdard users don't have excute access.

```
$su - student
$sh testuser.sh
```

```
the user is not root
```

[root@station1]# vi ifscript2

```
echo -n  Enter your no.:
read N

if [ $N -lt 50 ]

then echo  The No. you entered is less than 50?

elif [ $N -ge 50 -a $N -lt 60 ]

then

echo  The No. you entered is between 50 and 60?

elif [ $N -ge 60 -a $N -le 100 ]

then

echo  The No. you entered is between 60 and 100?

else

echo  The No. you entered out of range
```

fi

USER HOME DIRECTORY BACKUP (WITH USER I/P)

==========

vim userbackup

```
#!/bin/bash
##taking  users home dir backup ##

read -p "Enter user name to take home dir backup : " u

if [ $u  != root ]
then
homedir=/home/$u
else
homedir=/root
fi

tar cvzf /tmp/$u-`date +%F`.tar.gz $homedir
```

WITHOUT USER INPUT HOMEDIR BACKUP

==========

#vim userbackup.sh

```
#!/bin/bash

if [ $USER  != root ]
then
homedir=/home/$USER
else
homedir=/root
fi

tar cvzf /tmp/$USER-`date +%F`.tar.gz $homedir
```

UPLOADING FILES TO FTP SERVER

==========

```
#configure ftp server(192.168.0.11) with upload feature and ftp user should have passwd "redhat"
#vim /etc/vsftpd/vsftpd.conf  in 27 uncomment
```

SERVER=192.168.0.11

USER=ftp

PASSW=redhat

tar czf /root/install.tar.gz install.log

lftp -u $USER:$PASSW $SERVER -e "cd pub; put install.tar.gz; exit"

POSITIONAL PARAMETERS

==========

==>positional parameters: $0,$1,$2,$3,$#,$@

=>the two primary ways to pass data to a shell script are either through command-line arguments (positional parameters) or via standard input

(user's keyboard or via redirection)

=> $# the variabele returns the number of arguments to a shell script.

=> $1,$2,$3 are the first , second , third arguments passed to the shell script.

=> $0 refers to the name of the executable

Wirte a shell script which should print "python" if the user is passing first argument as "perl" , and script should print "perl" if the user first argument is

"python" , And if the number of arguments are zero , script should comeout with 0 exit status. else script should print error message as

"Err: perl|python" with exit status 1.

#vim exampscript

```
if [ "$1" == "perl" ]
 then
  echo "python"
elif [ "$1" == "python" ]
 then
  echo "perl"
elif [ $# == 0 ]
 then
  exit 0
else
 echo "Err: perl | python"
exit 1
fi
```

FEW MORE SIMILAR EXAMPLES:

==========

#vim examscript1

```
#!/bin/bash
if [ "$1" == "perl" ]
 then
```

```
  echo "python"
elif [ "$1" == "python" ]
 then
  echo "perl"
elif [ $# == 0 ]
 then
 echo "Err: perl | python" >&2
 exit 1
else
  exit 0
fi
```

#vim examscript2

```
#!/bin/bash
if [ "$1" == "perl" ]
 then
  echo "python"
elif [ "$1" == "python" ]
 then
  echo "perl"
else
 echo "Err: perl | python"  >&2
exit 1
fi
```

#vim examscript3

```
#!/bin/bash
if [ "$1" == "perl" ]
 then
  echo "python"
elif [ "$1" == "python" ]
 then
  echo "perl"
else
  exit 0
fi
```

Understanding uniq command usage

==========

uniq eleminates duplicates

#cat junk2.data

```
Linux
Debian
```

Redhat

Debian

Redhat

#sort junk2.data | uniq

cut command

---

in a file if there are multiple colums/fields using cut we can list only the required fields/columns. but

need to specify the delimeter(field separator)

#cat /etc/passwd | cut -d: -f3

-d: --> is to specify field separator

or

#cat /etc/passwd | cut -d: -f 3

or

#cat /etc/passwd | cut -d ':' -f3

or

#cat /etc/passwd | cut -d':' -f3

create a file with test name

#cat > test

---

linux redhat

redhat linux

laxma mensah

sriinfotech off_shore

#cat test | cut -d ' ' -f2   or   #cat test | cut -d' ' -f2

---

o/p will be

redhat

linux

mensah

off_shore

---

note: make sure in test file space in between   fields are separated with eqaul space that too only

one spacebar

if field seaparator is "," then

#cat test | cut -d ',' -f2

tr command

==========

==>tr character translation command , which is very useful

example:]# echo FILE1 | tr A-Z a-z

---

o/p will be:

file1

#tr --help

-s, --squeeze-repeats

echo FILEE1 | tr -s A-Z

---

o/p will be

FILE1

#echo FILEE1 | tr -s A-Z | tr A-Z a-z

---

o/p will be:

file1

OR

#echo FILEE1 | tr -s A-Z | tr [:upper:] [:lower:]

---

o/p will be:

file1

Note: in between upper and lower sets there is space & use --help page don't memorize

ONE EXAMPLE:

#mkdir /tmp2

#cd /tmp2

touch FILE{1..10}

#vim script3

#!/bin/bash

#Date 8 aug 2014

#Purpose: Illustrate using tr in a script to convert upper to lower filenames

```
for i  in  `ls -A`
do
newname=`echo $i  | tr  A-Z a-z `
mv  $i  $newname    # otherwise it will just print not rename
done
```

#chmod +x script3

#./script3

#ls

you can notice that all files in /tmp2 directory wich were in upper case got translated to lower case. after script3 execution. including script3 too.

to include if and elif syntax

Here what we want is , all files which are present /tmp2 should be renamed from UPPER CASE to lower case except script3

#vim script3

```
-------------------------------------

myscript=script3


for i  in  `ls -A`

do

if [ $i == $myscript ]

  then

  echo "i can't rename myself"

elif  [ $i != $myscript ]

  then

  newname=`echo $i | tr A-Z a-z`

  mv $i  $newname

fi

done

-----------------------------------------------------

#./script3



read command

-------------

#read  a


#echo $a


#read -p  "Enter the value of a:"  a
```

#echo $a

read -p "Enter the user name :" U

#read -sp   "Enter the password for U:" P

#echo $P

#read firstname lastname

#echo $firname $lastname

#read -n 3   answere   ==> read number of character 3 before existing, output will stored in variable

answere

yes

#echo $answere

yes

set command usage:

#TESTVAR=1000

#set | grep TEST

#unset TEST --> to remove temporarly defined variable

#mkdir test1 test2 test3

#rm -rf test*

$mkdir test{1,2,3}

#rm -rf test*

[

#mkdir test{1,2,3}{4,5,6}

---

o/p will be like

test14 test24 test34

test15 test25 test35

test16 test26 test36

---

command substitution:

---

command substitution allows us to store a command o/p into a variables

for command substitution we can go for either `` or $() #(openparanthesis) close paranthesis

#etcdir=`ls -l /etc/`

#echo $etcdir ==>o/p will be ugly so we can fix it

#echo "$etcdir" ==> currect o/p

OR

#ectdir1=$(ls -l /etc)

#echo $etcdir1

#echo "$etcdir1"

one more example:

#etclinecount=`ls -A /etc/ | wc -l`

#echo $etclinecount

EXAMPELS:

Write a shell script which should tell whether apache is running or not

```
# vim     script4
-------------------------------------------------
#purpose usage of command substitution

netstat  -ant  | grep :80  &>  /dev/null

APACHESTATUS=`echo $?`

if [ "$APACHESTATUS" -eq  0 ]
then
   echo  "Apache is Up and Running"
   else
   echo  "Apache is not Running"
fi

-------------------------------------------------------
#chmod +x  script4
#./script4
```